

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6 по теме «Процессы. Работа
с процессами»
по дисциплине
«Операционные системы»**

Выполнил студент группы МО-32/2 _____ В.И.Яценко

Направление подготовки 02.03.03 Математическое обеспечение и
администрирование информационных систем
Курс 3

Отчет принял _____ А.А.Полупанов

Краснодар
2025 г.

1. Дерево процессов в Linux

Для просмотра таблицы процессов в CMD мы использовали команду tasklist, в PowerShell командлет Get-Process, а в Linux для этого предназначена утилита ps. И начнем мы с наиболее часто используемых ключей aux:

```
user1@dc-1:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 103244 11652 ?        Ss   сен27   0:13 /sbin/init
root         2  0.0  0.0      0     0 ?        S    сен27   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        I<   сен27   0:00 [rcu_gp]
root         4  0.0  0.0      0     0 ?        I<   сен27   0:00 [rcu_par_gp]
root         6  0.0  0.0      0     0 ?        I<   сен27   0:00 [kworker/0:0H-kblockd]
root         9  0.0  0.0      0     0 ?        I<   сен27   0:00 [mm_percpu_wq]
root        10  0.0  0.0      0     0 ?        S    сен27   0:00 [rcu_tasks_rude_]
root        11  0.0  0.0      0     0 ?        S    сен27   0:00 [rcu_tasks_trace]
root        12  0.0  0.0      0     0 ?        S    сен27   0:00 [ksoftirqd/0]
root        13  0.0  0.0      0     0 ?        I    сен27   0:06 [rcu_sched]
root        14  0.0  0.0      0     0 ?        S    сен27   0:06 [migration/0]
root        15  0.0  0.0      0     0 ?        S    сен27   0:00 [idle_inject/0]
root        16  0.0  0.0      0     0 ?        S    сен27   0:00 [rcu_hb/0]
```

Рисунок 1 – просмотр таблиц процессов

С помощью ключей -ef выведем ту же информацию, но с идентификатором родителя:

```
user1@dc-1:~$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1      0  0 сен27 ?        00:00:13 /sbin/init
root         2      0  0 сен27 ?        00:00:00 [kthreadd]
root         3      2  0 сен27 ?        00:00:00 [rcu_gp]
root         4      2  0 сен27 ?        00:00:00 [rcu_par_gp]
root         6      2  0 сен27 ?        00:00:00 [kworker/0:0H-kblockd]
root         9      2  0 сен27 ?        00:00:00 [mm_percpu_wq]
root        10      2  0 сен27 ?        00:00:00 [rcu_tasks_rude_]
root        11      2  0 сен27 ?        00:00:00 [rcu_tasks_trace]
root        12      2  0 сен27 ?        00:00:00 [ksoftirqd/0]
root        13      2  0 сен27 ?        00:00:06 [rcu_sched]
root        14      2  0 сен27 ?        00:00:06 [migration/0]
root        15      2  0 сен27 ?        00:00:00 [idle_inject/0]
```

Рисунок 2 – просмотр таблиц процессов с помощью ключа -ef

Назначение использованных нами ключей следующее:

— ключ -e — отобразить процессы всех пользователей (то же самое, что ключ -a);

— ключ -f — вывести больше доступной информации, например, идентификатор родителя.

2.Адресное пространство

Обычно процессы могут аллоцировать весь доступный объем памяти, и ничего настраивать дополнительно не требуется. Выполните команду `ulimit -a`, и вы увидите, что для «virtual memory» установлено значение «unlimited»:

```
sa@astra:~$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 7524
max locked memory       (kbytes, -l) 65536
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 7524
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
sa@astra:~$
```

Рисунок 3 – команда `ulimit -a`

3. Сигналы для процессов в Linux

Для отправки сигналов из командной строки предназначена утилита `kill` (от англ. убить). Название утилиты объясняется тем, что без дополнительных уточнений она отправляет сигнал `SIGTERM`, который как раз и означает завершение работы. Для того чтобы отправить какой-то другой сигнал, нужно указать цифровой код сигнала - или его имя с помощью ключа `-s` (`--signal`):

Запускаем калькулятор в фоновом режиме и завершаем процесс с выбранным PID 1843 из прошлой команды:

```
sa@astra:~$ kcalc &
[1] 1877
sa@astra:~$ kill -SIGTERM 1877
```

Рисунок 4 – запуск калькулятора

Запускаем команду через `nohup`:

```
sa@astra:~$ nohup kcalc &
[2] 1884
[1]   Завершено      kcalc
sa@astra:~$ nohup: ввод игнорируется, вывод добавляется в 'nohup.out'
```

Рисунок 5 – nohup

4.Задания, интерактивный и фоновый режимы работы процесса

Просмотр фоновых заданий выполняется командой `jobs`. У этой команды есть следующие ключи:

Ключ `-l` – выводит список процессов заданий с PID процесса и информацией по умолчанию.

Ключ `-n` – выводит только те процессы, статус которых был изменен с последнего оповещения.

Ключ `-p` – выводит список только PID фоновых процессов.

Ключ `-r` – выводит только запущенные фоновые процессы.

Ключ `-s` – выводит только остановленные фоновые процессы. Давайте запустим три фоновых процесса и посмотрим список заданий:

```
sa@astra:~$ kcalc &
[2] 1902
[1]   Завершено      nohup kcalc
sa@astra:~$ kcalc &
[3] 1908
sa@astra:~$ kcalc &
[4] 1913
sa@astra:~$ jobs
[2]   Запущен      kcalc &
[3]- Запущен      kcalc &
[4]+ Запущен      kcalc &
sa@astra:~$
```

Рисунок 6 – запуск фоновых процессов

5.Содержимое /proc/PID/

```
sa@astra:~$ sudo ls /proc/1
[sudo] пароль для sa:
arch_status  coredump_filter  gid_map  mounts  pagemap  setgroups  task
attr         cpu_resctrl_groups  io       mountstats  patch_state  smaps      timens_offsets
autogroup    cpuset           limits   net       personality  smaps_rollup  timers
auxv         cwd              loginuid ns        projid_map  stack      timerslack_ns
cgroup       environ          map_files numa_maps  root        stat        uid_map
clear_refs   exe              maps     oom_adj   sched       statm       wchan
cmdline      fd               mem      oom_score  schedstat   status
comm         fdinfo           mountinfo oom_score_adj  sessionid   syscall
sa@astra:~$
```

Рисунок 7 – содержимое /proc/PID/

Файл `/proc/cmdline` – в нем содержится строка запуска процесса

```
sa@astra:~$ cat /proc/1/cmdline && echo
/sbin/init
sa@astra:~$
```

Рисунок 8 – `/proc/cmdline`

Файл `/proc/exe` – символическая ссылка, ведущая к полному пути до исполняемого файла. Полный путь необходим для однозначного понимания, какой именно файл был запущен.

```
sa@astra:~$ sudo ls -l --color=always /proc/1/exe
lrwxrwxrwx 1 root root 0 дек  7 14:11 /proc/1/exe -> /usr/lib/systemd/systemd
sa@astra:~$
```

Рисунок 9 – файл `/proc/exe`

Файл `/proc/cwd` – текущий рабочий каталог процесса PID1.

```
sa@astra:~$ sudo ls -l --color=always /proc/1/cwd
lrwxrwxrwx 1 root root 0 дек  7 14:40 /proc/1/cwd -> /
sa@astra:~$
```

Рисунок 10 – `/proc/cwd`

Файл `/proc/environ` – окружение процесса, создающее контекст его выполнения.

```
sa@astra:~$ sudo cat /proc/1/environ && echo
SHLVL=1HOME=/init=/sbin/initTERM=linuxBOOT_IMAGE=/boot/vmlinuz-5.15.0-33-genericdrop
usr/sbin:/bin:/usr/binPWD=/rootmnt=/root
sa@astra:~$
```

Рисунок 11 – `/proc/environ`

Каталог `/proc/fd/` – дескрипторы открытых файлов (от англ. file descriptors). Каждый файл в `fd/` представляет собой символическую ссылку и имеет числовое имя. Это не номер айноды в файловой системе, это уникальный номер в пределах процесса, который позволяет процессу обращаться к конкретным файлам, с которыми он работает. Дескрипторы 0, 1 и 2 – это стандартные ввод, вывод и вывод ошибок.

```

sa@astra:~$ sudo ls -l /proc/1/fd --color=always
итого 0
lrwx----- 1 root root 64 дек  7 14:11  0 -> /dev/null
lrwx----- 1 root root 64 дек  7 14:11  1 -> /dev/null
lrwx----- 1 root root 64 дек  7 14:46 10 -> 'anon_inode:[eventpoll]'
lrwx----- 1 root root 64 дек  7 14:46 100 -> 'socket:[20657]'
lrwx----- 1 root root 64 дек  7 14:46 101 -> 'socket:[20337]'
lrwx----- 1 root root 64 дек  7 14:46 102 -> 'socket:[20266]'
lrwx----- 1 root root 64 дек  7 14:46 103 -> 'socket:[20102]'
lrwx----- 1 root root 64 дек  7 14:46 104 -> 'socket:[20033]'
lrwx----- 1 root root 64 дек  7 14:46 106 -> 'socket:[19744]'
lrwx----- 1 root root 64 дек  7 14:46 109 -> 'socket:[19524]'
lr-x----- 1 root root 64 дек  7 14:46 11 -> anon_inode:inotify
lrwx----- 1 root root 64 дек  7 14:46 110 -> 'socket:[19465]'
lrwx----- 1 root root 64 дек  7 14:46 111 -> 'socket:[19404]'
lrwx----- 1 root root 64 дек  7 14:46 112 -> 'socket:[17953]'
lrwx----- 1 root root 64 дек  7 14:46 113 -> 'socket:[19335]'
lrwx----- 1 root root 64 дек  7 14:46 114 -> /run/initctl
lrwx----- 1 root root 64 дек  7 14:46 115 -> 'socket:[19336]'

```

Рисунок 12 – каталог /proc/fd/

Файл /proc/io – содержит сведения об объемах данных, прочитанных и записанных процессом в хранилище информации

```

sa@astra:~$ sudo cat /proc/1/io
rchar: 672510034
wchar: 11621006
syscr: 493344
syscw: 370834
read_bytes: 728373248
write_bytes: 8192000
cancelled_write_bytes: 2732032
sa@astra:~$

```

Рисунок 13 – /proc/io

Файл /proc/limits – отображает различные ограничения процесса, установленные конфигурационным файлом /etc/security/limits.conf и командой ulimit.

```

sa@astra:~$ sudo cat /proc/1/limits
Limit                  Soft Limit             Hard Limit              Units
Max cpu time           unlimited              unlimited              seconds
Max file size          unlimited              unlimited              bytes
Max data size          unlimited              unlimited              bytes
Max stack size         8388608               unlimited              bytes
Max core file size     0                     unlimited              bytes
Max resident set       unlimited              unlimited              bytes
Max processes          7524                  7524                  processes
Max open files         1048576               1048576               files
Max locked memory      67108864              67108864              bytes
Max address space      unlimited              unlimited              bytes
Max file locks         unlimited              unlimited              locks
Max pending signals    7524                  7524                  signals
Max msgqueue size      819200                819200                bytes
Max nice priority      0                     0
Max realtime priority  0                     0
Max realtime timeout   unlimited              unlimited              us
sa@astra:~$

```

Рисунок 14 – /proc/limits

Файл /proc/maps – физические адреса страниц памяти, используемые в данный момент.

```

sa@astra:~$ sudo cat /proc/1/maps | head
57b86f113000-57b86f141000 r--p 00000000 08:01 396408      /usr/lib/sy
stemd/systemd
57b86f141000-57b86f25c000 r-xp 0002e000 08:01 396408      /usr/lib/sy
stemd/systemd
57b86f25c000-57b86f2b1000 r--p 00149000 08:01 396408      /usr/lib/sy
stemd/systemd
57b86f2b1000-57b86f2ea000 r--p 0019d000 08:01 396408      /usr/lib/sy
stemd/systemd
57b86f2ea000-57b86f2eb000 rw-p 001d6000 08:01 396408      /usr/lib/sy
stemd/systemd
57b87128b000-57b871418000 rw-p 00000000 00:00 0          [heap]
754e10000000-754e10021000 rw-p 00000000 00:00 0
754e10021000-754e14000000 ---p 00000000 00:00 0
754e150b5000-754e150b6000 ---p 00000000 00:00 0
754e150b6000-754e158b6000 rw-p 00000000 00:00 0
sa@astra:~$

```

Рисунок 15 – /proc/maps

Файл /proc/sched – отображение текущих значений переменных планировщика процессов, необходимых для корректной работы планировщика SFC.

```

sa@astra:~$ sudo cat /proc/1/sched
systemd (1, #threads: 1)
-----
se.exec_start                :          2321920.602759
se.vruntime                  :           782.388449
se.sum_exec_runtime          :          999.477333
se.nr_migrations             :              0
nr_switches                  :           2775
nr_voluntary_switches        :           1020
nr_involuntary_switches      :           1755
se.load.weight               :         1048576
se.avg.load_sum              :            323
se.avg.runnable_sum          :          331666
se.avg.util_sum              :          331666
se.avg.load_avg              :              7
se.avg.runnable_avg          :              7
se.avg.util_avg              :              7
se.avg.last_update_time      :        2321920602112
se.avg.util_est.ewma         :              8
se.avg.util_est.enqueued     :              0
uclamp.min                   :              0
uclamp.max                   :           1024
effective uclamp.min         :              0
effective uclamp.max         :           1024
policy                       :              0
prio                         :           120
clock-delta                  :            33
mm->numa_scan_seq            :              0
numa_pages_migrated          :              0
numa_preferred_nid           :             -1
total_numa_faults            :              0
current_node=0, numa_group_id=0
numa_faults node=0 task_private=0 task_shared=0 group_private=0 group_shared=0

```

Рисунок 16 – /proc/sched

Файлы /proc/stat и /proc/status – близкие по своей сути, отражают основные сведения о процессе. Файл stat - машиночитаемый формат, а status - человекочитаемый. Большинство параметров, выводимых утилитой ps, она получает из структур данных, аналогичных тем, которые отображаются в этих файлах.

```

sa@astra:~$ sudo cat /proc/1/stat
1 (systemd) S 0 1 1 0 -1 4194560 10707 224115 171 41580 6 93 304 2770 20 0 1 0 4 1055006
72 2864 18446744073709551615 96449649184768 96449650343801 140737100612336 0 0 0 6711731
23 4096 1260 1 0 0 17 0 0 0 0 0 0 96449650695888 96449650925888 96449684090880 140737100
619595 140737100619606 140737100619606 140737100619757 0
sa@astra:~$

```

Рисунок 17 – /proc/stat


```
sa@astra:~$ sudo cat /proc/1/status
Name:      systemd
Umask:     0000
State:     S (sleeping)
Tgid:      1
Ngid:      0
Pid:       1
PPid:      0
TracerPid: 0
Uid:       0      0      0      0
Gid:       0      0      0      0
FDSize:    128
Groups:
NSTgid:    1
NSpid:     1
NSpgid:    1
NSSid:     1
VmPeak:    167648 kB
VmSize:    103028 kB
VmLck:     0 kB
VmPin:     0 kB
VmHWM:     11456 kB
VmRSS:     11456 kB
RssAnon:   2664 kB
RssFile:   8792 kB
RssShmem:  0 kB
```

Рисунок 18 – /proc/status

6. Содержимое /proc

Ниже представлено содержимое каталога /proc/. В выводе команды все каталоги с PID заменены на {PID 1,2,...,N} для лучшего восприятия. Файлы, хранящиеся в корне каталога /proc/, характеризуют состояние и характеристики всего вычислительного узла и ядра ОС.

```

sa@astra:~$ ls /proc
1      1315  1437  1830  3      74      974      iomem      schedstat
10     1316  1453  1835  320    746    977      ioports    scsi
100    1323  1477  19      354    75      996      irq        self
1000   1324  15     1902   377    76      acpi      kallsyms   slabinfo
101    1332  16     1908   379    77      asound    kcore      softirqs
1076   1333  1684   1913   4       78      bootconfig keys        stat
11     1341  169    1923   412    79      buddyinfo key-users   swaps
111    1352  17     1975   413    80      bus       kmsg       sys
1115   1361  170    1976   414    82      cgroups   kpagecgrou sysrq-trigger
1135   1372  171    2       421    84      cmdline   kpagecount sysvipc
116    1373  172    20     422    85      consoles  kpageflags thread-self
117    1374  173    21     432    87      cpuinfo    loadavg     timer_list
1190   1376  174    215    435    88      crypto     locks        tty
1191   1377  175    216    440    9       devices    mdstat       uptime
12     1380  176    2189   443    90      diskstats meminfo      version
1205   1381  177    22     495    91      dma        misc         version_signature
122    1386  1774   23     504    92      driver     modules      vmallocinfo
1268   1388  178    24     580    93      dynamic_debug mounts        vmstat
1270   1389  1782   25     591    94      execdomains mtrr         zoneinfo
1293   1390  1785   26     6       95      fb         net
1298   1397  1793   265    626    956     filesystems pagetypeinfo
1299   1398  1796   278    72     968     fs         partitions
13     14     18     287    73     97      interrupts pressure
sa@astra:~$

```

Рисунок 19 – каталог /proc

Файл /proc/cmdline – список параметров, которые были переданы ядру при загрузке.

```

sa@astra:~$ cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-5.15.0-33-generic root=UUID=eba34003-adda-47f3-a50d-8b9513eb81d
c ro parsec.max_ilev=63 quiet net.ifnames=0
sa@astra:~$

```

Рисунок 20 – /proc/cmdline

Файл /proc/cpuinfo – сведения о всех установленных процессорах.

```

sa@astra:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 140
model name     : 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
stepping       : 1
microcode      : 0xffffffff
cpu MHz        : 2148.432
cache size     : 8192 KB
physical id    : 0
siblings       : 1
core id        : 0
cpu cores      : 1
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 22
wp             : yes

```

Рисунок 21 – /proc/cpuinfo

Файл `/proc/diskstats` – статистика операций со всеми дисками. Для каждого диска своя строка.

```
sa@astra:~$ cat /proc/diskstats
 7      0 loop0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      1 loop1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      2 loop2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      3 loop3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      4 loop4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      5 loop5 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      6 loop6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 7      7 loop7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 8      0 sda 27349 8680 2051082 33846 3815 7311 105584 1
```

Рисунок 22 – `/proc/diskstats`

Файл `/proc/meminfo` – отображение информации о состоянии памяти. Предоставляет больше параметров, чем утилита `free`.

```
sa@astra:~$ free
              total        used         free       shared    buff/cache   available
Mem:           2025456      434484        475316         21892       1115656       1404548
Swap:           998396           0         998396

sa@astra:~$ cat /proc/meminfo
MemTotal:        2025456 kB
MemFree:         475316 kB
MemAvailable:    1404584 kB
Buffers:         53132 kB
Cached:          996188 kB
SwapCached:      0 kB
Active:          282180 kB
Inactive:        1067684 kB
Active(anon):    6560 kB
Inactive(anon):  315876 kB
Active(file):    275620 kB
Inactive(file):  751808 kB
Unevictable:     0 kB
Mlocked:         0 kB
SwapTotal:       998396 kB
SwapFree:        998396 kB
Dirty:           0 kB
Writeback:       0 kB
AnonPages:      300588 kB
```

Рисунок 23 – `/proc/meminfo`

Файл `/proc/devices` – перечень устройств в системе.

```
sa@astra:~$ cat /proc/devices
Character devices:
1 mem
4 /dev/vc/0
4 tty
4 ttyS
5 /dev/tty
5 /dev/console
5 /dev/ptmx
5 ttyprintk
6 lp
7 vcs
10 misc
13 input
21 sg
29 fb
89 i2c
99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
204 ttyMAX
226 drm
```

Рисунок 24 – /proc/devices

Файл /proc/filesystem – перечень файловых систем, поддерживаемых ядром ОС.

```
sa@astra:~$ cat /proc/filesystems
nodev sysfs
nodev tmpfs
nodev bdev
nodev proc
nodev cgroup
nodev cgroup2
nodev cpuset
nodev devtmpfs
nodev configfs
nodev debugfs
nodev tracefs
nodev securityfs
nodev sockfs
nodev bpf
nodev pipefs
nodev ramfs
nodev hugetlbfs
nodev devpts
nodev ext2
```

Рисунок 25 – /proc/filesystem

Файл /proc/mounts – перечень смонтированных файловых систем (формат данных, аналогичен файлу /etc/fstab).

```

sa@astra:~$ cat /proc/mounts
sysfs /sys sysfs rw,nosuid,nodev,noexec,relatime 0 0
proc /proc proc rw,nosuid,nodev,noexec,relatime 0 0
udev /dev devtmpfs rw,nosuid,relatime,size=963204k,nr_inodes=240801,mode=755,inode64 0 0
devpts /dev/pts devpts rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000 0 0
tmpfs /run tmpfs rw,nosuid,noexec,relatime,size=202548k,mode=755,inode64 0 0
/dev/sda1 / ext4 rw,relatime,errors=remount-ro 0 0
parsecfs /parsecfs parsecfs rw,sync,relatime 0 0
securityfs /sys/kernel/security securityfs rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /dev/shm tmpfs rw,nosuid,nodev,inode64 0 0
tmpfs /run/lock tmpfs rw,nosuid,nodev,noexec,relatime,size=5120k,inode64 0 0
tmpfs /sys/fs/cgroup tmpfs ro,nosuid,nodev,noexec,mode=755,inode64 0 0
cgroup2 /sys/fs/cgroup/unified cgroup2 rw,nosuid,nodev,noexec,relatime,nsdelegate 0 0
cgroup /sys/fs/cgroup/systemd cgroup rw,nosuid,nodev,noexec,relatime,xattr,name=systemd
0 0
pstore /sys/fs/pstore pstore rw,nosuid,nodev,noexec,relatime 0 0
bpf /sys/fs/bpf bpf rw,nosuid,nodev,noexec,relatime,mode=700 0 0
cgroup /sys/fs/cgroup/cpu,cpuacct cgroup rw,nosuid,nodev,noexec,relatime,cpu,cpuacct 0 0
cgroup /sys/fs/cgroup/perf_event cgroup rw,nosuid,nodev,noexec,relatime,perf_event 0 0
cgroup /sys/fs/cgroup/net_cls,net_prio cgroup rw,nosuid,nodev,noexec,relatime,net_cls,ne
t_prio 0 0
cgroup /sys/fs/cgroup/rdma cgroup rw,nosuid,nodev,noexec,relatime,rdma 0 0

```

Рисунок 26 – /proc/mounts

Файл /proc/modules – список подгруженных модулей ядра. Ядро Linux монолитное, но модульное, и мы можем управлять тем, какие модули подгружать, а какие нет, тем самым определяя функционал запускаемой ОС.

```

sa@astra:~$ cat /proc/modules
vboxvideo 36864 0 - Live 0x0000000000000000 (OE)
drm_ttm_helper 16384 1 vboxvideo, Live 0x0000000000000000 (E)
intel_rapl_msr 20480 0 - Live 0x0000000000000000 (E)
intel_rapl_common 32768 1 intel_rapl_msr, Live 0x0000000000000000 (E)
crct10dif_pclmul 16384 1 - Live 0x0000000000000000 (E)
crc32_pclmul 16384 0 - Live 0x0000000000000000 (E)
ghash_clmulni_intel 16384 0 - Live 0x0000000000000000 (E)
joydev 32768 0 - Live 0x0000000000000000 (E)
aesni_intel 376832 0 - Live 0x0000000000000000 (E)
crypto_simd 16384 1 aesni_intel, Live 0x0000000000000000 (E)
cryptd 24576 2 ghash_clmulni_intel,crypto_simd, Live 0x0000000000000000 (E)
intel_cstate 20480 0 - Live 0x0000000000000000 (E)
snd_intel8x0 49152 3 - Live 0x0000000000000000 (E)
snd_ac97_codec 155648 1 snd_intel8x0, Live 0x0000000000000000 (E)
ac97_bus 16384 1 snd_ac97_codec, Live 0x0000000000000000 (E)
snd_pcm 135168 2 snd_intel8x0,snd_ac97_codec, Live 0x0000000000000000 (E)
input_leds 16384 0 - Live 0x0000000000000000 (E)
serio_raw 20480 0 - Live 0x0000000000000000 (E)
vboxguest 421888 6 - Live 0x0000000000000000 (OE)
snd_timer 40960 1 snd_pcm, Live 0x0000000000000000 (E)
snd 102400 10 snd_intel8x0,snd_ac97_codec,snd_pcm,snd_timer, Live 0x0000000000000000 (E)
soundcore 16384 1 snd, Live 0x0000000000000000 (E)
mac_hid 16384 0 - Live 0x0000000000000000 (E)
vhci_hcd 53248 0 - Live 0x0000000000000000 (E)
usbip_host 45056 0 - Live 0x0000000000000000 (E)
usbip_core 40960 2 vhci_hcd,usbip_host, Live 0x0000000000000000 (E)
parport_pc 53248 0 - Live 0x0000000000000000 (E)

```

Рисунок 27 – /proc/modules

Файл /proc/swaps – список разделов подкачки.

```

sa@astra:~$ cat /proc/swaps
Filename                                Type                                Size                                Used                                P
riority
/dev/sda5                               partition                            998396                             0                                  -
2
sa@astra:~$

```

Рисунок 28 – /proc/swaps

Файл /proc/version – версия ядра ОС.

```
sa@astra:~$ cat /proc/version
Linux version 5.15.0-33-generic (builder@build5) (gcc (AstraLinuxSE 8.3.0-6) 8.3.0, GNU
ld (GNU Binutils for AstraLinux) 2.31.1) #astra2+ci96 SMP Fri Oct 28 18:23:37 UTC 2022
sa@astra:~$
```

Рисунок 29 – /proc/version

Каталог /sys/kernel/ содержит набор файлов, которые позволяют нам оперативно без перезагрузки изменять параметры ядра ОС (но таких параметров не очень много). Поэтому часть файлов в этом каталоге доступна для записи, и через эти интерфейсы мы можем управлять ядром. Некоторые из них будут рассмотрены в лабораторной, посвящённой ядру ОС.

```
sa@astra:~$ ls /proc/sys/kernel/
acct                               perf_event_max_stack
acpi_video_flags                  perf_event_mlock_kb
auto_msgmni                       perf_event_paranoid
bootloader_type                   pid_max
bootloader_version                poweroff_cmd
bpf_stats_enabled                 print-fatal-signals
cad_pid                           printk
cap_last_cap                      printk_delay
core_pattern                      printk_devkmsg
core_pipe_limit                   printk_ratelimit
core_uses_pid                     printk_ratelimit_burst
ctrl-alt-del                      pty
dmesg_restrict                    random
domainname                        randomize_va_space
firmware_config                   real-root-dev
ftrace_dump_on_oops               sched_autogroup_enabled
ftrace_enabled                    sched_cfs_bandwidth_slice_us
hardlockup_all_cpu_backtrace      sched_child_runs_first
hardlockup_panic                  sched_deadline_period_max_us
hostname                          sched_deadline_period_min_us
hotplug                           sched_energy_aware
hung_task_all_cpu_backtrace       sched_rr_timeslice_ms
hung_task_check_count              sched_rt_period_us
hung_task_check_interval_secs     sched_rt_runtime_us
hung_task_panic                   sched_schedstats
```

Рисунок 30 – /proc/kernel

7. Просмотр процессов

Для просмотра процессов есть три основные консольные утилиты (ps, top и htop) и одна графическая (системный монитор). Утилита ps позволяет получить снимок текущего состояния всех или заданных процессов и их параметров.

```
sa@astra:~$ ps
  PID TTY          TIME CMD
 1830 pts/1        00:00:00 bash
 1902 pts/1        00:00:00 kcalc
 1908 pts/1        00:00:00 kcalc
 1913 pts/1        00:00:00 kcalc
 2304 pts/1        00:00:00 ps
sa@astra:~$
```

Рисунок 31 – утилита ps

8. Утилита ps

Для сортировки процессов можно воспользоваться параметром `--sort`, например, `ps aux --sort=%mem` или `ps aux --sort=%cpu`.

```
sa@astra:~$ ps aux --sort=%mem | tail -n 3
sa      1386  0.0  3.4 784032 69088 ?        Ssl  14:11   0:00 nm-applet
sa      1390  0.0  4.6 888132 94180 ?        Ssl  14:11   0:00 fly-search-panel --hidd
en
fly-dm   968  0.8  5.1 594792 103948 tty7      Ssl+ 14:11   0:27 /usr/lib/xorg/Xorg -br
-novtswitch -quiet -keeptty :0 vt7 -logfile /var/log/fly-dm/Xorg.%s.log -seat seat0 -aut
h /var/run/xauth/A:0-w60kJb
```

Рисунок 32 – утилита ps

Команда `ps -eo euser,ruser,suser,fuser,f,comm,label` выведет информацию об атрибутах EUID, RUID, SUID.

```
sa@astra:~$ ps -eo euser,ruser,suser,fuser,f,comm,label
EUSER   RUSER   SUSER   FUSER   F COMMAND      LABEL
root    root    root    root    4 systemd      0:63:0:0
root    root    root    root    1 kthreadd      0:63:0:0
root    root    root    root    1 rcu_gp        0:63:0:0
root    root    root    root    1 rcu_par_gp    0:63:0:0
root    root    root    root    1 kworker/0:0H-ev 0:63:0:0
root    root    root    root    1 mm_percpu_wq  0:63:0:0
root    root    root    root    1 rcu_tasks_rude_ 0:63:0:0
root    root    root    root    1 rcu_tasks_trace 0:63:0:0
root    root    root    root    1 ksoftirqd/0    0:63:0:0
root    root    root    root    1 rcu_sched      0:63:0:0
root    root    root    root    1 migration/0    0:63:0:0
root    root    root    root    1 idle_inject/0  0:63:0:0
root    root    root    root    1 cpuhp/0        0:63:0:0
root    root    root    root    5 kdevtmpfs      0:63:0:0
root    root    root    root    1 netns         0:63:0:0
root    root    root    root    1 inet_frag_wq   0:63:0:0
root    root    root    root    1 kauditd        0:63:0:0
root    root    root    root    1 khungtaskd     0:63:0:0
root    root    root    root    1 rcu_scheduler  0:63:0:0
```

Рисунок 33 – команда `ps -eo euser,ruser,suser,fuser,f,comm,label`

С помощью утилиты `ps` можно посмотреть и потоки. Воспользуйтесь командами `ps -eLf` и `ps axms`.


```
sa@astra:~$ ps -elf
F S UID          PID    PPID    C  PRI   NI   ADDR  SZ  WCHAN    STIME TTY          TIME CMD
4 S root           1        0  0  80    0   - 25757 -      14:11 ?          00:00:01 /sbin/init
1 S root           2        0  0  80    0   -    0 -      14:11 ?          00:00:00 [kthreadd]
1 I root           3        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [rcu_gp]
1 I root           4        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [rcu_par_gp]
1 I root           6        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [kworker/0:0H
1 I root           9        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [mm_percpu_wq
1 S root          10        2  0  80    0   -    0 -      14:11 ?          00:00:00 [rcu_tasks_ru
1 S root          11        2  0  80    0   -    0 -      14:11 ?          00:00:00 [rcu_tasks_tr
1 S root          12        2  0  80    0   -    0 -      14:11 ?          00:00:00 [ksoftirqd/0]
1 I root          13        2  0  80    0   -    0 -      14:11 ?          00:00:01 [rcu_sched]
1 S root          14        2  0  -40   - -    0 -      14:11 ?          00:00:00 [migration/0]
1 S root          15        2  0   9    - -    0 -      14:11 ?          00:00:00 [idle_inject/
1 S root          16        2  0  80    0   -    0 -      14:11 ?          00:00:00 [cpuhp/0]
5 S root          17        2  0  80    0   -    0 -      14:11 ?          00:00:00 [kdevtmpfs]
1 I root          18        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [netns]
1 I root          19        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [inet_frag_wq
1 S root          20        2  0  80    0   -    0 -      14:11 ?          00:00:00 [kauditd]
1 S root          21        2  0  80    0   -    0 -      14:11 ?          00:00:00 [khungtaskd]
1 S root          22        2  0  80    0   -    0 -      14:11 ?          00:00:00 [oom_reaper]
1 I root          23        2  0  60   -20 -    0 -      14:11 ?          00:00:00 [writeback]
```

Рисунок 34 – потоки с помощью ps

```
sa@astra:~$ ps axmu
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.5 103028 11456 ?        -   14:32   0:01 /sbin/init
root           -  0.0  - - - - -      Ss   14:32   0:01 -
root           2  0.0  0.0    0     0 ?        -   14:32   0:00 [kthreadd]
root           -  0.0  - - - - -      S   14:32   0:00 -
root           3  0.0  0.0    0     0 ?        -   14:32   0:00 [rcu_gp]
root           -  0.0  - - - - -      I<  14:32   0:00 -
root           4  0.0  0.0    0     0 ?        -   14:32   0:00 [rcu_par_gp]
root           -  0.0  - - - - -      I<  14:32   0:00 -
root           6  0.0  0.0    0     0 ?        -   14:32   0:00 [kworker/0:0H-events_highpr
root           -  0.0  - - - - -      I<  14:32   0:00 -
root           9  0.0  0.0    0     0 ?        -   14:32   0:00 [mm_percpu_wq]
root           -  0.0  - - - - -      I<  14:32   0:00 -
root          10  0.0  0.0    0     0 ?        -   14:32   0:00 [rcu_tasks_rude_]
root           -  0.0  - - - - -      S   14:32   0:00 -
root          11  0.0  0.0    0     0 ?        -   14:32   0:00 [rcu_tasks_trace]
root           -  0.0  - - - - -      S   14:32   0:00 -
root          12  0.0  0.0    0     0 ?        -   14:32   0:00 [ksoftirqd/0]
root           -  0.0  - - - - -      S   14:32   0:00 -
root          13  0.0  0.0    0     0 ?        -   14:32   0:02 [rcu_sched]
root           -  0.0  - - - - -      I   14:32   0:02 -
root          14  0.0  0.0    0     0 ?        -   14:32   0:01 [migration/0]
root           -  0.0  - - - - -      S   14:32   0:01 -
```

Рисунок 35 – потоки с помощью ps

Для фильтрации вывода команды ps используйте встроенные опции: -123 или 123, -p "1 2" – вывод информации по PID процессу(ам).

```
sa@astra:~$ ps -1
  PID TTY          STAT       TIME COMMAND
    1 ?           Ss          0:01 /sbin/init
sa@astra:~$ ps 1
  PID TTY          STAT       TIME COMMAND
    1 ?           Ss          0:01 /sbin/init
sa@astra:~$ ps -p "1 2"
  PID TTY          TIME CMD
    1 ?           00:00:01 systemd
    2 ?           00:00:00 kthreadd
sa@astra:~$
```


Рисунок 36 – фильтрация

-C <имя-команды> – вывод информации по процессам, имя команды которых соответствует указанному, например, ps -C bash.

```
sa@astra:~$ ps -C bash
  PID TTY          TIME CMD
 1796 pts/0        00:00:00 bash
 1830 pts/1        00:00:00 bash
sa@astra:~$
```

Рисунок 37 – вывод информации по процессам

Используя опцию -o pid, можно вывести только колонку с PID найденных процессов, а используя -o pid=, только PID без названия колонки.

```
sa@astra:~$ ps -C bash -o pid
PID
1796
1830
sa@astra:~$ ps -C bash -o pid=
1796
1830
sa@astra:~$
```

Рисунок 38 – опция -o pid

Для дополнительной фильтрации можно использовать утилиту grep, например, ps aux | grep tty

```
sa@astra:~$ ps aux | grep tty
root      626  0.0  0.0   5808   1932 tty1      Ss+  14:32   0:00 /sbin/agetty -o -p -- \u --
noclear  tty1  linux
fly-dm    968  0.5  5.4 602536 111320 tty7      Ssl+ 14:32   1:21 /usr/lib/xorg/Xorg -br -nov
tswitch -quiet -keeptty :0 vt7 -logfile /var/log/fly-dm/Xorg.%s.log -seat seat0 -auth /var/r
un/xauth/A:0-w60kJb
sa        3929  0.0  0.0   6096    812 pts/1      S+   18:56   0:00 grep tty
sa@astra:~$
```

Рисунок 39 – утилита grep

9. Утилита pstree

С помощью утилиты pstree можно вывести дерево процессов. Для её использования необходим пакет psmisc sudo apt install psmisc.

```

sa@astra:~$ pstree
systemd├─NetworkManager├─dhclient──{dhclient}
│                       └─2*[{NetworkManager}]
│
│   └─3*[VBoxClient─VBoxClient─3*[{VBoxClient}]]
│       └─VBoxClient─VBoxClient─4*[{VBoxClient}]
│           └─VBoxDRMClient─4*[{VBoxDRMClient}]
│               └─VBoxService─8*[{VBoxService}]
│
│   └─agetty
│   └─alsactl
│   └─astra-orientati─2*[{astra-orientati}]
│   └─at-spi2-registr─2*[{at-spi2-registr}]
│   └─auditd──{auditd}
│   └─avahi-daemon─avahi-daemon
│   └─cron
│   └─cupsd
│   └─2*[dbus-daemon]
│   └─dbus-launch
│   └─fly-baloorunner─2*[{fly-baloorunner}]
│   └─fly-dm├─Xorg─5*[{Xorg}]
│           └─fly-dm─fly-wm├─astra-event-wat──{astra-event-wat}
│                           └─at-spi-bus-laun──dbus-daemon
│                               └─3*[{at-spi-bus-laun}]
│                                   └─baloo_file─3*[{baloo_file}]
│                                       └─fly-cups-watch──{fly-cups-watch}
│                                           └─fly-notify-osd─2*[{fly-notify-osd-}]
│                                               └─fly-reflex-serv─3*[{fly-reflex-serv}]
│                                                   └─fly-search-pane─10*[{fly-search-pane}]
│                                                       └─kmix─3*[{kmix}]
│                                                           └─kscreend─2*[{kscreend}]
│                                                               └─nm-applet─7*[{nm-applet}]
│                                                                   └─org_kde_powerde─5*[{org_kde_powerde}]
│                                                                       └─polkit-kde-auth─4*[{polkit-kde-auth}]
│                                                                           └─ssh-agent
│
│   └─fly-getexe
│   └─fly-smb-watcher──{fly-smb-watcher}

```

Рисунок 40 – утилита pstree